

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

Sorting III

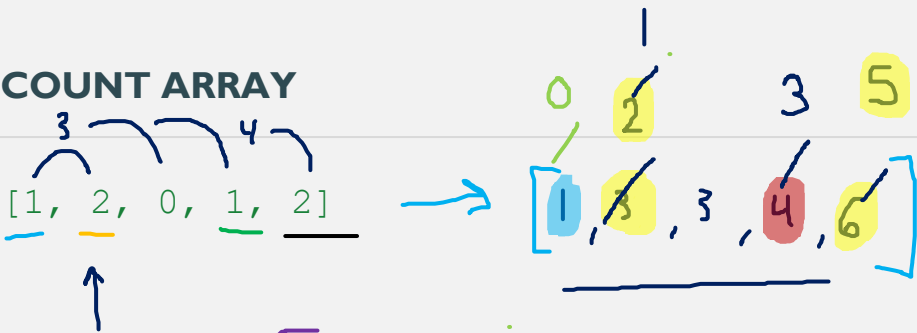
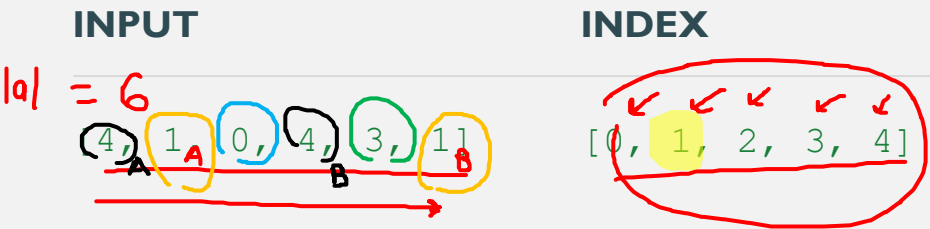


COUNTING SORT

Assume every element is an integer in the range 0 to k-1. Then the key is already an array index.

Py.
list.sort()

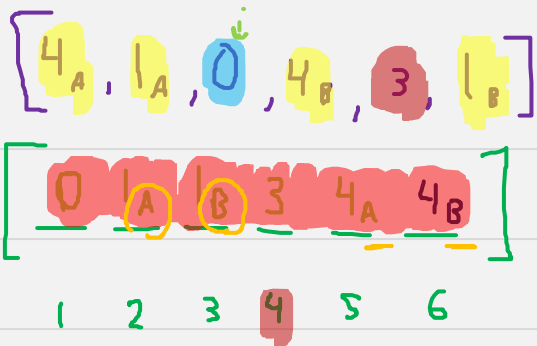
Java
array.sort()



PASS

ARRAY SO FAR

1	[0]
2	[0, 1, 1]
3	[0, 1, 1]
4	[0, 1, 1, 3]
5	[0, 1, 1, 4, 4]



Stable!



COUNTING SORT

Assume every element is an integer in the range 0 to $k-1$. Then the key is already an array index.

```
def counting_sort(arr, k):
```

```
    counts = [0] * k
```

```
    for x in arr:
```

```
        counts[x] += 1
```

```
    for d in range(1, k):
```

```
        counts[d] += counts[d - 1]
```

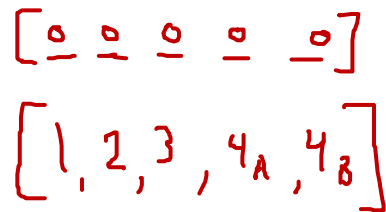
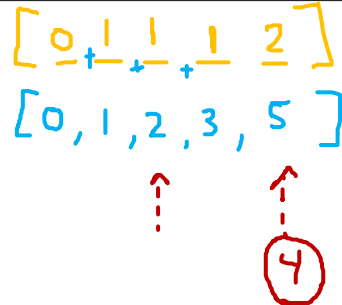
```
    out = [0] * len(arr)
```

```
    for x in reversed(arr):
```

```
        counts[x] -= 1
```

```
        out[counts[x]] = x
```

```
    return out
```



THREE PASSES, ALL LINEAR

Count each value: $\Theta(n)$. Turn counts into running totals: $\Theta(k)$. Place each element at its computed index: $\Theta(n)$. Total $\Theta(n + k)$, with no comparison anywhere.

WHY THE LAST LOOP RUNS BACKWARD

The running total is the index just past the last slot for that value. Walking the input in reverse and decrementing hands out slots back to front, so equal keys keep their original order. Counting sort is stable, and the next algorithm depends on it.



RADIX SORT RUNS COUNTING SORT ONCE PER DIGIT

One pass per digit, least significant first, each pass a stable counting sort on that digit alone. Nothing is ever compared.

170 → 3 digits

PASS	SORT KEY	ARRAY AFTER THE PASS
start	unsorted	170 045 075 090 802 024 002 066
1	<u>ones digit</u>	170 090 802 002 024 045 075 066
2	<u>tens digit</u>	802 002 024 045 066 170 075 090
3	<u>hundreds digit</u>	002 024 045 066 075 090 170 802



LINEAR, WITH CONDITIONS ATTACHED

0 1 1 1 0 0

1 1 1 1 1 1

0...9

$$T(n) = \underline{\Theta(d \cdot (n + k))} \quad d = \text{number of digits}, k = \text{size of the digit alphabet}$$

WHEN IT WINS

Fixed width integer keys, a small base, and d that does not grow with n . Then d and k are constants and the total is $\Theta(n)$.

Sorting n 32-bit integers in base 256 is 4 passes, no matter how large n gets.

WHEN IT LOSES

Keys that are not integers, keys whose width grows with n , or a base large enough that k dominates.

It also allocates $\Theta(n + k)$ per pass and is never in place, so a memory-bound workload can lose on the constants alone.

Radix $\rightarrow$ lower bound



does no comparisons



TIMSORT. WHAT PYTHON ACTUALLY RUNS.



REAL DATA IS ALREADY PARTLY SORTED

sorted() and list.sort() in Python, and Arrays.sort() on objects in Java, all run Powersort (a variation of Timsort). It is a merge sort that looks at the input first.

01 Find the runs

[0, 1, 1, 1, 1, 2, 2, 2, 6, 4, 3, 4, 9, 19, 90, 2, 4]

Scan left to right for maximal stretches that are already ascending, or strictly descending. A descending run is reversed in place, which is why the reversal test must be strict: reversing a stretch of equal keys would break stability.

02 Extend short runs to minrun

minrun is chosen between 32 and 64 so that the number of runs is close to a power of two. A run shorter than minrun is grown with binary insertion sort, which is $\Theta(n^2)$ but on a block of at most 64 elements with excellent locality.

03 Merge runs, never blindly

Runs are pushed on a stack and merged in an order that keeps their lengths balanced. Merging is the same procedure from last lecture, and it is where the $\Theta(n)$ extra space goes.

↓
LO5

Insertion $O(n^2) \rightarrow |32|, |64|$

Quicksort
if (len(arr) < 128)
Insertion

$$O(n^2)$$

$[1, 2, 3] \rightarrow [4, 5, 6]$

$$60 \times 120$$

THE STACK INVARIANT

The invariant forces merges to happen between runs of similar length, which is what keeps the total at $\Theta(n \log n)$.

GALLOPING MODE

When one run is entirely smaller than the other, this turns $\Theta(n)$ comparisons into $\Theta(\log n)$.

THE 2015 BUG

It was found by formally verifying the implementation, not by testing it.

BEST CASE

WORST CASE

STABLE

EXTRA SPACE

ADAPTIVE

 $\Theta(n \log n)$

Yes

 $\Theta(n)$

Yes



Optimality

Time sort

Adaptive

✓	✓	✓	✓
---	---	---	---

Merge

Radix

Comp | Fast?

✓	x
x	✓

CLOSING THOUGHTS

Memory

Quick

Merge

Run | In-place

x	✓
✓	x

$n \log n$

